Rapport TP1 — Analyse Statique (SAST) et Sécurité des Dépendances (SCA)

Auteur : Hanine Khemir — INSAT **Cours :** DevSecOps **Date :** 13 octobre 2026 **Dépôt :** https://github.com/HanineKhemir/devsecops-tp1-sast-sca

---

1. Objectif et démarche « shift-left »

Le principe **shift-left** consiste à déplacer les contrôles de sécurité le plus tôt possible dans le cycle de développement — au moment où l'on écrit le code, pas après la mise en production. Corriger une faille pendant le développement coûte beaucoup moins cher (en temps et en risque) que la corriger une fois l'application déployée.

Ce TP applique deux familles d'outils automatisés qui incarnent cette approche :

Famille	Question posée	Outils utilisés
SAST (Static Application Security Testing)	« Mon **code** contient-il des failles ? »	Bandit, SonarQube
SCA (Software Composition Analysis)	« Mes **dépendances** contiennent-elles des failles connues (CVE) ? »	OSV / pip-audit, OWASP Dependency-Check

L'application support est **VulnBank**, une petite application web Flask volontairement vulnérable, écrite pour ce TP. L'historique Git raconte toute la démarche :

```
baseline      → application vulnérable (avant tout scan)
fix(sast)     → correctifs d'analyse statique
fix(sca)      → mise à jour des dépendances vulnérables
```

---

2. Mise en place de l'environnement

- **Application** : Flask (`vulnbank/`), base SQLite, routes de connexion, consultation de comptes, commentaires, export et taux de change.
- **Conteneurisation** : `Dockerfile` construisant l'image de l'application.

```
docker build -t vulnbank:baseline .
docker run --rm -e HOST=0.0.0.0 -p 5000:5000 vulnbank:baseline
# http://localhost:5000  (comptes de test : alice/password, admin/admin)
```

Tous les scanners ont été exécutés via Docker, sans rien installer sur le poste.

---

3. Analyse statique (SAST)

3.1 Outils et exécution

Bandit (analyseur statique spécialisé Python) :

```
docker run --rm -v "$PWD":/src -w /src python:3.11-slim \
  sh -c "pip install bandit && bandit -r vulnbank"
```

SonarQube (plateforme SAST avec tableau de bord) :

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community
# scan via sonar-scanner-cli, résultats sur http://localhost:9000
```

3.2 État initial — les vulnérabilités détectées

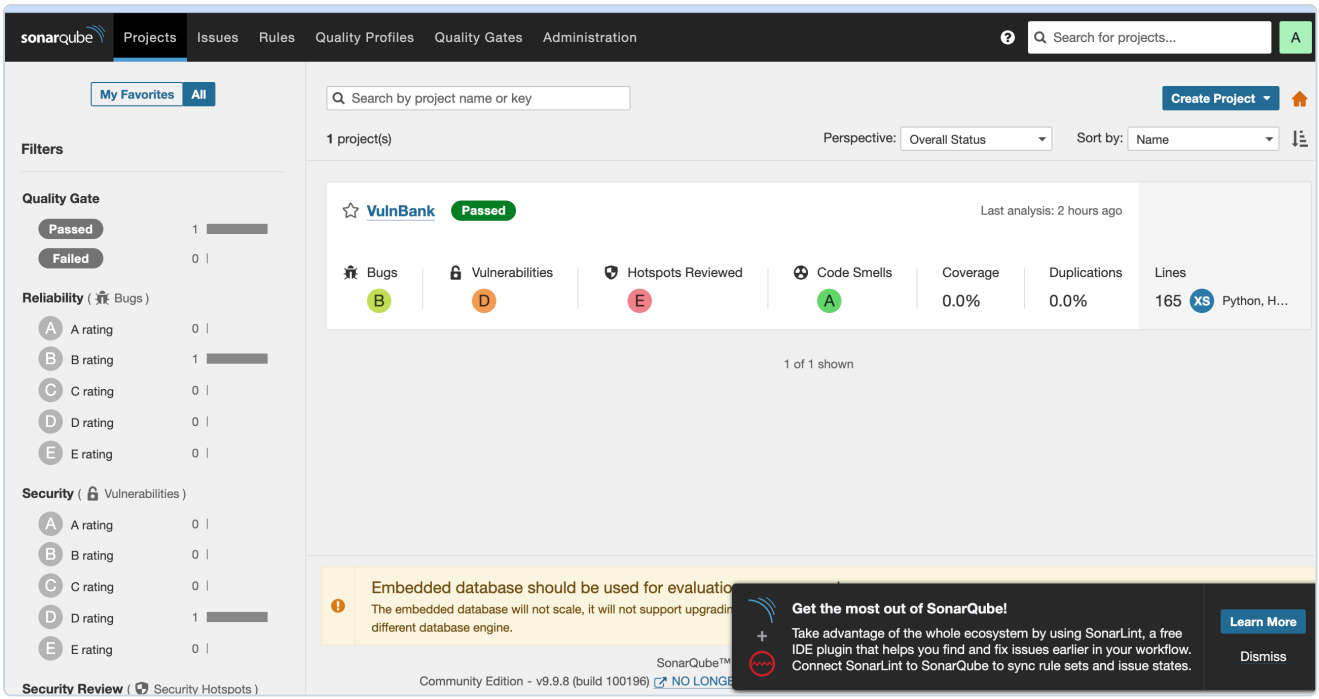
Bandit a signalé **9 problèmes** sur le code initial :

Ligne	Règle	Vulnérabilité	Gravité
11	B105	Clé secrète en dur dans le code	Low
19	B506	`yaml.load()` non sécurisé (exécution de code)	Medium
23	B324	Hachage des mots de passe en **MD5**	High
41	B608	**Injection SQL** (route `/login`)	Medium
66	B608	**Injection SQL** (route `/accounts`)	Medium
88	B605	**Injection de commande** (`os.system`)	High
94	B501	Validation du certificat TLS désactivée (`verify=False`)	High
100	B201	Flask lancé avec `debug=True`	High
100	B104	Écoute sur toutes les interfaces (`0.0.0.0`)	Medium

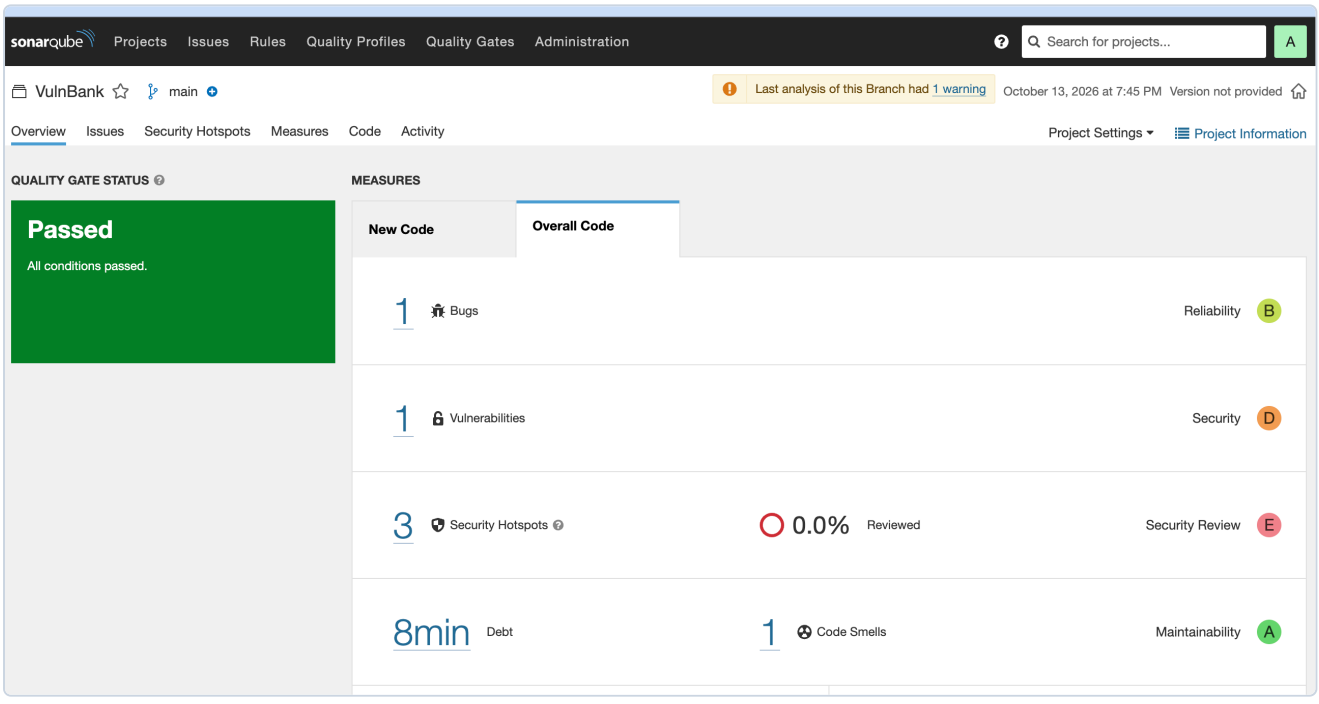
SonarQube a signalé de son côté **1 vulnérabilité** et **3 points sensibles (hotspots)** :

- Vulnérabilité `python:S4830` — validation du certificat TLS désactivée (ligne 94)
- Hotspots : hachage MD5 (ligne 23) et CSRF (×2)

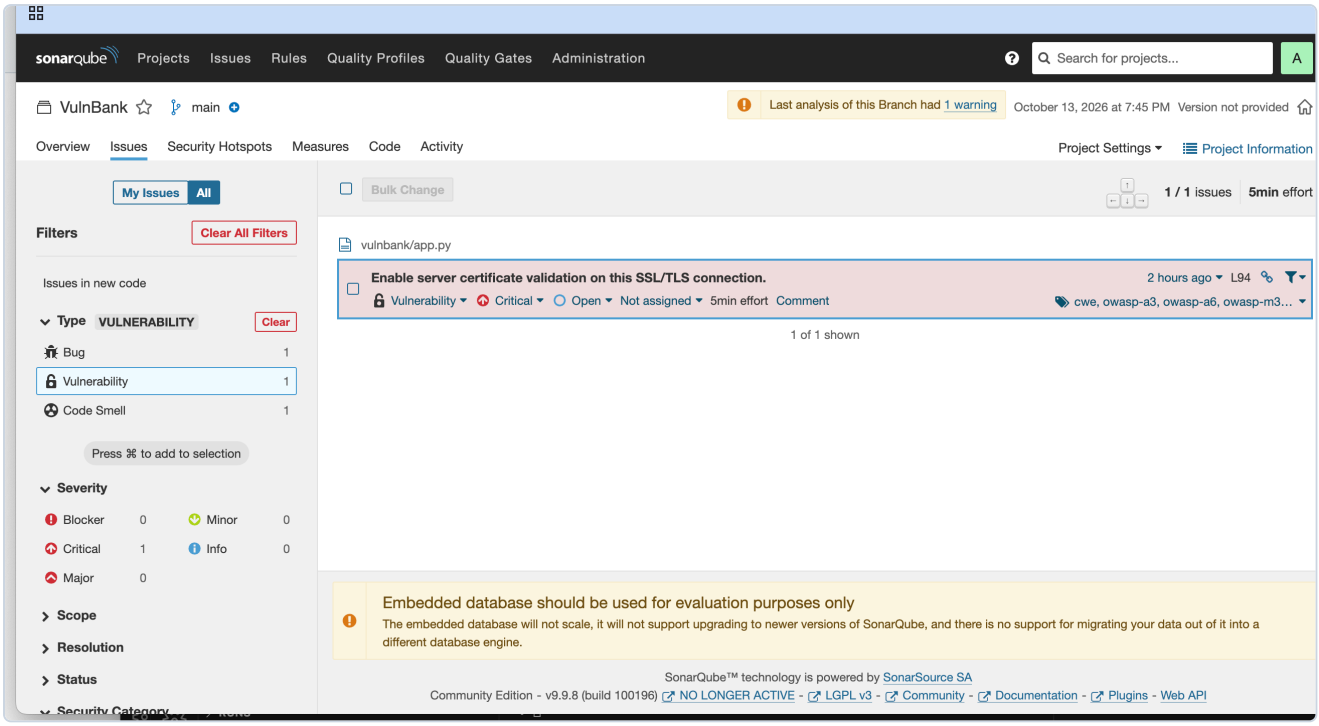
Capture — liste des projets (avant) : sécurité notée **D**, hotspots **E**.



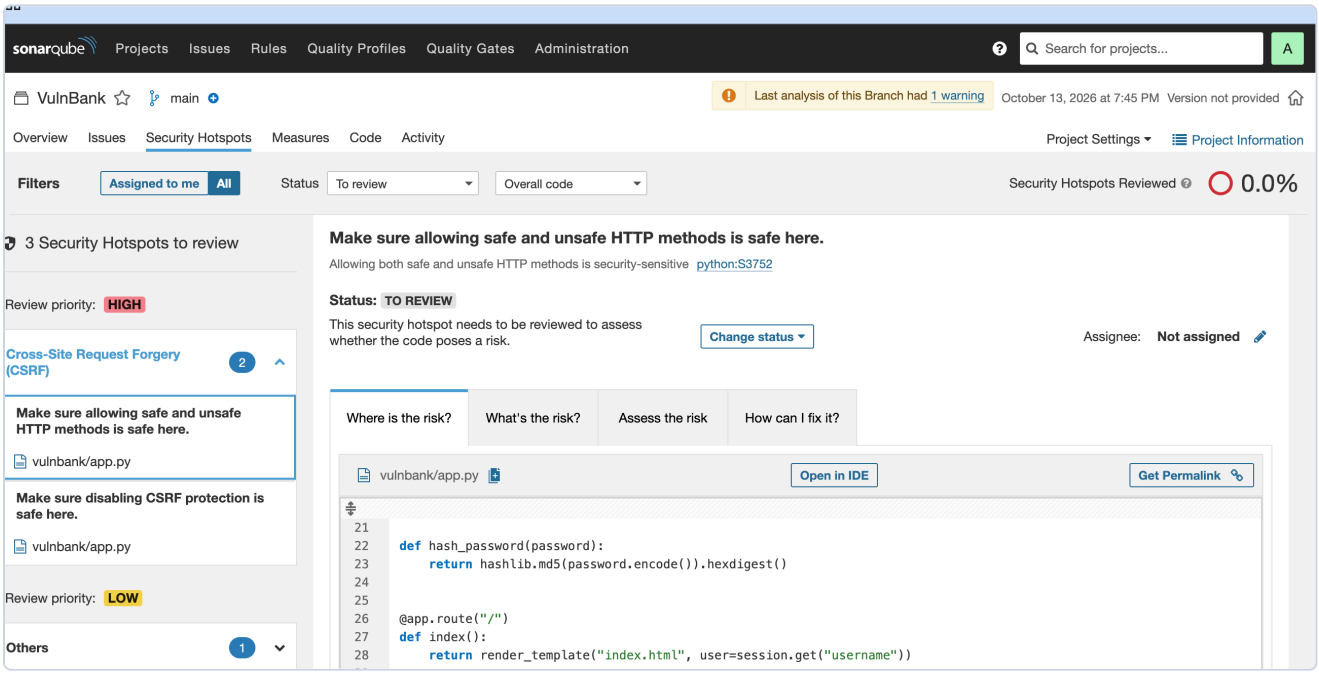
Capture — vue d'ensemble du projet (avant) : 1 vulnérabilité (Security **D**), 3 hotspots (Security Review **E**).



Capture — détail de la vulnérabilité (avant) : « Enable server certificate validation on this SSL/TLS connection » à `app.py:94`, gravité **Critical**, étiquettes OWASP (`owasp-a3` , `owasp-a6`).



Capture — points sensibles (avant) : 3 hotspots à réviser (CSRF ×2 + MD5), 0 % révisé.



> **Observation importante (bon point pour la démarche) :** SonarQube **Community Edition ne fait pas d'analyse de teinte (taint analysis)**. Il ne détecte donc **pas** l'injection SQL ni l'injection de commande — celles-ci apparaissent uniquement dans Bandit. C'est la raison pour laquelle une chaîne CI/CD sérieuse combine plusieurs outils SAST : aucun scanner ne voit tout.

3.3 Correctifs appliqués (commit `fix(sast)`)

Vulnérabilité	Correctif
Injection SQL (×2)	Requêtes **paramétrées** (`?` placeholders) au lieu de f-strings
Injection de commande	Suppression de `os.system` ; export via l'API SQLite + validation du nom de compte
XSS	`render_template` (auto-échappement Jinja2) au lieu de `render_template_string`
Hachage MD5	`werkzeug.security` (pbkdf2) avec sel par utilisateur
TLS désactivé	Validation du certificat réactivée (`verify=True` par défaut)
`yaml.load`	`yaml.safe_load`
Secret en dur	Lu depuis une variable d'environnement
`debug=True` / bind `0.0.0.0`	Contrôlés par variables d'environnement, valeurs sûres par défaut

Exemple — correction de l'injection SQL dans `/login` :

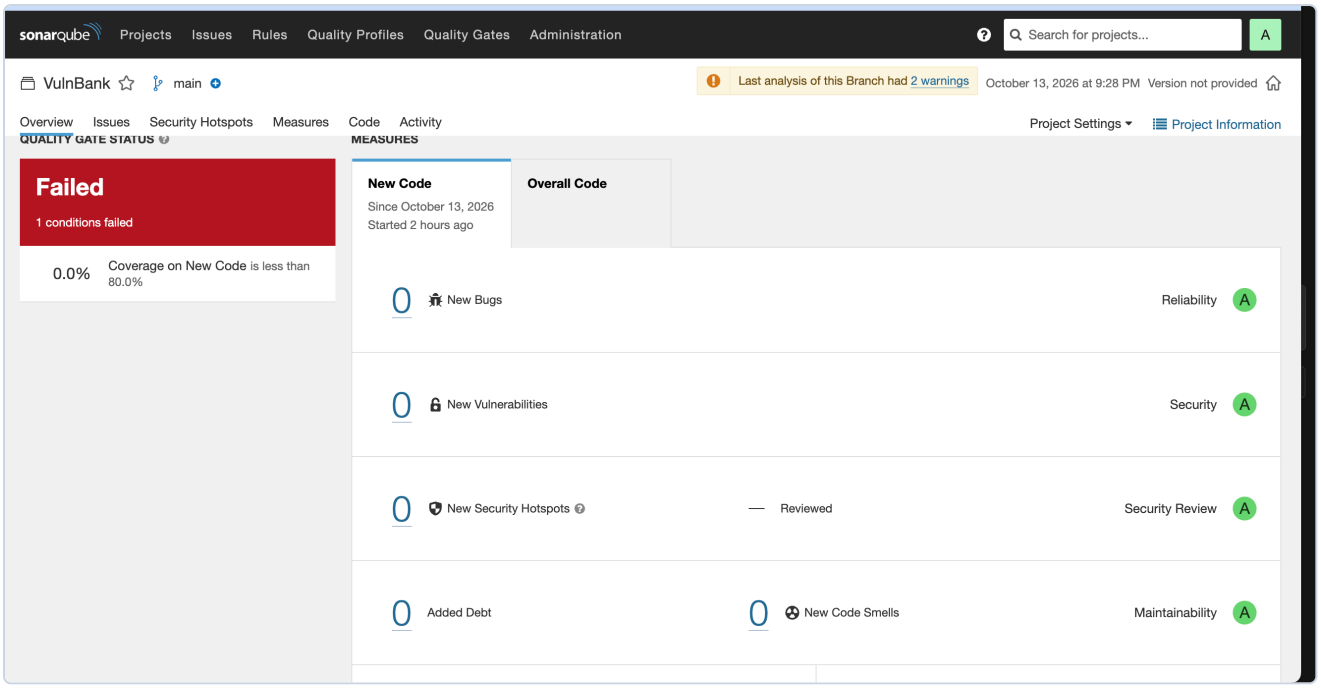
```
# AVANT (vulnérable) — la saisie devient de la syntaxe SQL
query = f"SELECT ... WHERE username = '{username}' AND password = '{password}'"
row = conn.execute(query).fetchone()

# APRÈS (corrigé) — la saisie est un paramètre lié, jamais du code SQL
row = conn.execute(
    "SELECT id, username, role, password FROM users WHERE username = ?",
    (username,),
).fetchone()
```

3.4 Résultat après correction

- **Bandit : 9 → 0 problème.**
- **SonarQube : Security D → A, 0 vulnérabilité** (la vulnérabilité `S4830` passe au statut *Closed* / *Fixed*). Le hotspot MD5 disparaît.

Capture — après correction : sécurité notée **A**, **0 nouvelle vulnérabilité**, 0 bug, 0 hotspot sur le nouveau code (re-scan de 21 h 28).



> **Note sur le « Quality Gate : Failed » visible sur la capture** — l'échec provient **uniquement** de la condition *Coverage < 80 %* (il n'y a pas de tests unitaires dans ce TP), **pas** d'un problème de sécurité. Toutes les dimensions de sécurité sont notées **A** avec **0 vulnérabilité**. On pourrait lever cet échec en ajoutant des tests ou en ajustant le Quality Gate, ce qui sort du périmètre du TP.

Vérification des deux vulnérabilités phares corrigées :

```
# L'injection SQL ne contourne plus l'authentification
curl -X POST localhost:5000/login --data-urlencode "username=x' OR '1'='1" --data "pa
#  -> HTTP 401 (rejeté)   [avant : connexion réussie]
```

---

4. Analyse des dépendances (SCA)

4.1 Outils et exécution

Les dépendances étaient épinglées sur des versions anciennes dans `requirements.txt`. Scan via **pip-audit / OSV** (base de données de vulnérabilités), outil fiable pour l'écosystème Python :

```
# Interrogation de la base OSV pour chaque dépendance épinglée
pip-audit -r requirements.txt
```

> **Note outillage :** OWASP Dependency-Check (cité dans le sujet) est orienté Java/Maven et détecte mal les paquets Python d'un `requirements.txt`. OSV / pip-audit donne des résultats exploitables pour Python — c'est un point de « bon outil pour le bon écosystème » à retenir.

4.2 État initial — dépendances vulnérables

7 paquets vulnérables, 44 CVE uniques (84 avis au total) :

Paquet	Version initiale	CVE phare
Jinja2	2.10	**CVE-2019-10906** — évasion du bac à sable via `str.format_map`
Werkzeug	0.14.1	CVE-2024-34069 — RCE via le debugger
Flask	0.12.2	CVE-2023-30861 — divulgation de session via cache
urllib3	1.23	CVE-2019-11236 — injection CRLF
requests	2.19.1	CVE-2023-32681 — fuite de l'en-tête Proxy-Authorization
certifi	2018.11.29	CVE-2022-23491 — autorités de certification compromises
idna	2.7	CVE-2024-3651 — déni de service (complexité quadratique)

4.3 Correctif et pourquoi la mise à jour est nécessaire

Mise à jour vers des versions corrigées (commit `fix(sca)`), et passage de l'image de base **Python 3.9 → 3.11** pour pouvoir atteindre les dernières versions corrigées :

Paquet	Avant	Après
Jinja2	2.10	3.1.6
Werkzeug	0.14.1	3.1.9
Flask	0.12.2	3.1.3
urllib3	1.23	2.8.0
requests	2.19.1	2.33.0
certifi	2018.11.29	2024.8.30
idna	2.7	3.15

Explication de la CVE phare — Jinja2 2.10 / CVE-2019-10906 :

Jinja2 est le moteur de templates de Flask. Les versions ≤ 2.10 contiennent une **évasion du bac à sable** : via la méthode `str.format_map`, un attaquant capable d'influencer un template peut **sortir de l'environnement restreint** et accéder à des objets Python arbitraires, ouvrant la voie à l'**exécution de code** (RCE). Pour une application qui rend du contenu à partir d'entrées utilisateur, c'est une faille critique.

Pourquoi mettre à jour est indispensable (et non optionnel) :

1. **La faille est publique et connue.** Une CVE est référencée dans des bases de données publiques (NVD, OSV, GitHub Advisory). N'importe quel attaquant — et n'importe quel

scanner automatisé — connaît la version vulnérable et la méthode d'exploitation. Garder la version ancienne, c'est exposer une porte dont tout le monde a le plan.
2. **Le correctif existe déjà.** La version 2.10.1 corrige précisément cette évasion. Se mettre à jour est le moyen le plus rapide et le moins risqué de fermer la faille — aucun contournement maison n'est nécessaire.
3. **Le risque est transitif.** Même si *mon* code n'utilise pas directement `str.format_map`, la bibliothèque, elle, l'expose. La sécurité d'une application dépend de **toute** sa chaîne de dépendances, pas seulement du code écrit à la main.

4.4 Résultat après mise à jour

- **SCA : 84 → 0 avis, 0 paquet vulnérable.**
- L'application fonctionne toujours sur la pile mise à jour (Flask 3 / Werkzeug 3) : page d'accueil HTTP 200, connexion valide HTTP 302.

---

5. Conclusion — ce que le TP démontre

Phase	Avant	Après
SAST (Bandit)	9 problèmes	**0**
SAST (SonarQube)	Security **D**, 1 vulnérabilité	Security **A**, **0**
SCA (OSV / pip-audit)	7 paquets, 44 CVE	**0**

Le TP illustre concrètement l'approche **shift-left** :

- Les failles ont été trouvées **automatiquement**, directement sur le code et les dépendances, avant tout déploiement.
- Plusieurs outils sont **complémentaires** : Bandit voit les injections que SonarQube Community ne voit pas; OSV/pip-audit est le bon outil pour les dépendances Python.
- La SCA n'est **pas un contrôle ponctuel** : de nouvelles CVE paraissent chaque semaine. Elle doit tourner en continu dans la CI/CD pour rester efficace.

Annexe — reproduire les scans

```
git clone https://github.com/HanineKhemir/devsecops-tp1-sast-sca
cd devsecops-tp1-sast-sca

# Revenir à l'état vulnérable pour voir le « avant »
git checkout baseline

# SAST (Bandit)
docker run --rm -v "$PWD":/src -w /src python:3.11-slim \
  sh -c "pip install bandit && bandit -r vulnbank"

# SCA (pip-audit)
docker run --rm -v "$PWD":/src:ro -w /tmp python:3.11-slim \
  sh -c "cp /src/requirements.txt . && pip install pip-audit && pip-audit -r requirer

# Revenir à l'état corrigé pour voir le « après »
git checkout main
```

Les rapports bruts des scanners sont dans le dossier `reports/`.